

MINOR PROJECT REPORT

Industrial Engineering and Operations Research, IIT Delhi



MediShift

An Intelligent, Ratio-Aware Staff Rostering System for Hospitals

Machine Learning Forecasting and Mixed-Integer Optimisation
for Safe, Fair Hospital Staff Schedules

Seasonal Patient Load vs Staffing Gap (Target Ratio)

Prepared by: Ejisaya Naik (2025MEE2856)

Supervisor: Prof. Amber Srivastava

Project: MediShift (Android Application)

Report date: August 2026

A concise technical account of the problem, the architecture, and the mathematics behind MediShift's forecasting and scheduling engines.

• SUMMER INTERNSHIP/ PROJECT · SUMMER SEMESTER •

Abstract

Government and district hospitals are often short of doctors, nurses, pharmacists and lab technicians relative to patient demand. Understaffing puts patients at risk; inefficient staffing wastes the hours of the staff who are on duty. MediShift is an Android app that addresses both: it forecasts patient inflow for every shift, then converts that forecast into a staff roster using ratio-matching and mixed-integer optimisation, respecting rest rules, shift limits, and staff leave in real time.

This report covers MediShift in three chapters. Chapter 1 sets out the problem, the project's objectives, and the system architecture. Chapter 2 covers the forecasting engine: Ridge Regression, Gradient-Boosted Stumps, and Holt-Winters smoothing, with derivations. Chapter 3 covers how a forecast becomes a schedule: the two-ratio staffing model, the per-shift and person-level assignment algorithm, and automatic leave handling. Pseudocode, flowcharts and an algorithm summary table run throughout.

Keywords: hospital staff rostering, patient-to-staff ratio, forecasting, ridge regression, gradient boosting, Holt-Winters smoothing, mixed-integer linear programming, leave management.

Notation

The symbols below are used consistently across Chapters 2 and 3. A reader who is new to the mathematics can treat this page as a quick reference and return to it whenever a symbol reappears.

Symbol	Meaning
$y_i, \hat{y}_i$	Actual and predicted patient inflow for historical record i
$\mathbf{x}_i, \boldsymbol{\beta}$	Feature vector for record i ; the fitted coefficient vector
w_i	Recency weight given to historical record i
α	Ridge Regression's penalty strength
η	Gradient boosting's learning rate
L_t, T_t, S_t	Holt-Winters level, trend, and seasonal state at time t
δ	Cyclic weekday index (0 = Monday, ..., 6 = Sunday) used for the day-of-week feature
$\lambda_{\text{ridge}}, \lambda_{\text{boosted}}, \lambda_{\text{holt}}$	Inverse-MAE ensemble weights for each fitted model, within their combined 60% share
ϕ_d	Empirical day-of-week seasonality factor applied to the heuristic forecast for day d
p	Predicted patient count for a shift
g_c, t_c	Good ratio and Target ratio for staff category c
$\text{minSafe}_c, \text{ideal}_c$	Hard safety floor and aspirational headcount for category c
n_c, pool_c	Chosen headcount and available staff pool for category c
$x_{i,d,s}$	1 if staff member i works shift s on day d , else 0

Table 1: Notation used throughout this report.

Contents

Abstract	1
Notation	2
1 Introduction and System Architecture	4
1.1 Problem Statement	4
1.2 Background	4
1.3 Objectives and Scope	4
1.4 System Architecture	5
1.5 Data Source and Features	6
2 Machine Learning and Predictions	8
2.1 Forecasting Approach	8
2.2 Linear and Ridge Regression	8
2.3 Gradient-Boosted Decision Stumps	9
2.4 Time-Series Forecasting with Holt-Winters	9
2.5 Ensemble Blending and Evaluation	10
3 Staff Selection and Roster Optimisation	13
3.1 The Two-Ratio Staffing Model	13
3.2 Per-Shift Allocation and Person-Level Assignment	14
3.3 Leave Management and Real-Time Integration	17
3.4 Algorithms at a Glance	17
3.5 Future Work	17
Future Work	17
3.6 Conclusion	18
References	19

Chapter 1

Introduction and System Architecture

1.1 Problem Statement

Government and district hospitals often struggle to maintain a healthy ratio between patients and available staff. This directly affects patient satisfaction, since an overloaded staff member cannot give each patient proper attention, and it affects the staff themselves, who don't get adequate rest or a fair work culture when short-staffed. The problem this project sets out to solve is to optimize hospital staff scheduling so that patient satisfaction is protected while also ensuring staff get proper rest and a sustainable work culture — using ratio-based, demand-responsive staffing rather than fixed, one-size-fits-all schedules.

1.2 Background

The Google Deep Research platform is used to study and reviewed regarding typical safe ratios of different staff. During study it is found that safe ratios are around one doctor per 10–20 in-patients, one nurse per 4–6 patients in general wards, one pharmacist per 50–75 patients a day, and one lab technician per 30–40 samples a shift – while real reporting shows nurse-to-patient ratios stretching to 1:20 in some district hospitals, with specialist doctor vacancy rates above 50% in parts of the public system. The WHO's Workload Indicators of Staffing Need (WISN) methodology exists because this mismatch is structural, not a one-hospital problem.

Understaffing puts patients at risk; overstaffing every shift just moves the shortage elsewhere, since staff on an already well-covered shift are staff unavailable where they are actually needed. The right target is not "as many staff as possible" but a *ratio* the hospital trusts as safe, met as closely as the available staff allow.

MediShift builds this into two linked numbers: a **Good ratio**, the loosest ratio still clinically safe and treated as a hard floor, and a stricter **Target ratio**, the ideal the hospital aims for. Every staffing decision in the app, from a single shift's headcount to a full week's roster, rests on this pair.

1.3 Objectives and Scope

The project has four goals: forecast patient inflow per shift with genuine, data-fitted models rather than a fixed guess; convert that forecast into a safe staffing target via the Good/Target ratio pair; turn that target into a named weekly roster that respects hard rules (one shift per person per day, a weekly shift cap, mandatory rest after night shifts); and let approved leave update the scheduler automatically, with no manual recomputation. A quieter fifth goal runs throughout: every simplification made so the system can run entirely on a phone, with no server, is stated honestly rather than glossed over.

Each formula in this report is introduced in plain language first, so a non-technical reader can follow the reasoning, while the derivations and pseudocode give a technical reader enough to verify or reimplement the logic. Out of scope for this version: multi-department coordination

(one department’s roster at a time), skill-tier substitution within a category (a senior and junior nurse count equally), and integration with an external hospital information system – natural extensions once this core pipeline is validated in practice.

1.4 System Architecture

MediShift is built around one idea: there is only ever one copy of the truth about the hospital’s current state, and every screen simply displays it. Figure 1.1 shows the layered structure. The **presentation layer** holds the screens, grouped by role: login and app shell, the Doctor/Nurse day-to-day screens, the Operations Manager’s forecasting and roster-approval screens, and Admin screens. Every screen reads state and requests changes; none keeps a private copy.

Beneath that sits one component, **MediShiftViewModel**, holding every piece of live state (staff list, leave requests, latest forecast, latest optimisation result, generated roster) as observable state every screen subscribes to. Two engines do the reasoning: the **ML Forecasting Engine** (Chapter 2) predicts patient inflow, and the **Optimisation Engine** (Chapter 3) turns that prediction, with the ratio policy from Section 1.1, into a staffing plan and named roster. At the bottom, a single **Repository** is the only component that talks to the on-device **Room database**, so every data-access rule lives in one place.

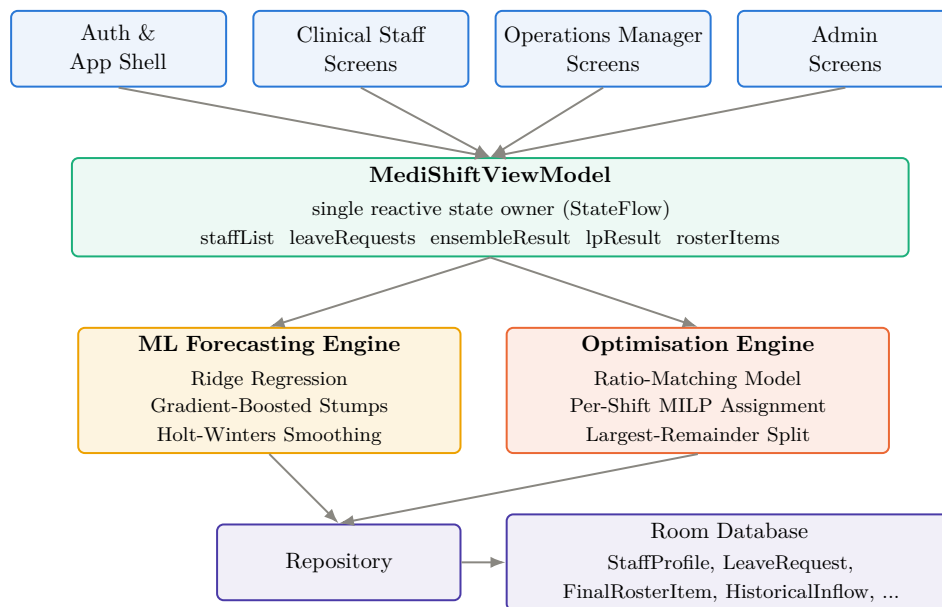


Figure 1.1: Layered architecture of MediShift. Every screen reads and writes state only through the single ViewModel, so a change anywhere, such as an approved leave request, is reflected everywhere at once, with no manual refresh.

The app is written in **Kotlin** with **Jetpack Compose** for its interface and the **MVVM** pattern, with **Room** providing a type-safe layer over an on-device SQLite database. Since Android cannot run Python natively, every model in Chapter 2 is implemented directly in Kotlin using the same underlying mathematics a data scientist would use in scikit-learn or statsmodels, not a shortcut approximation – each model’s trade-offs are discussed where it is introduced.

Figure 1.1 shows the architecture in the abstract; Figure 1.2 shows it on an actual device through one full cycle: an HR coordinator approving a candidate into the staff pool, an

Operations Manager running a forecast, computing a safe staffing target, and releasing the roster as a shared PDF, and a doctor seeing that week’s confirmed shifts on their own screen.

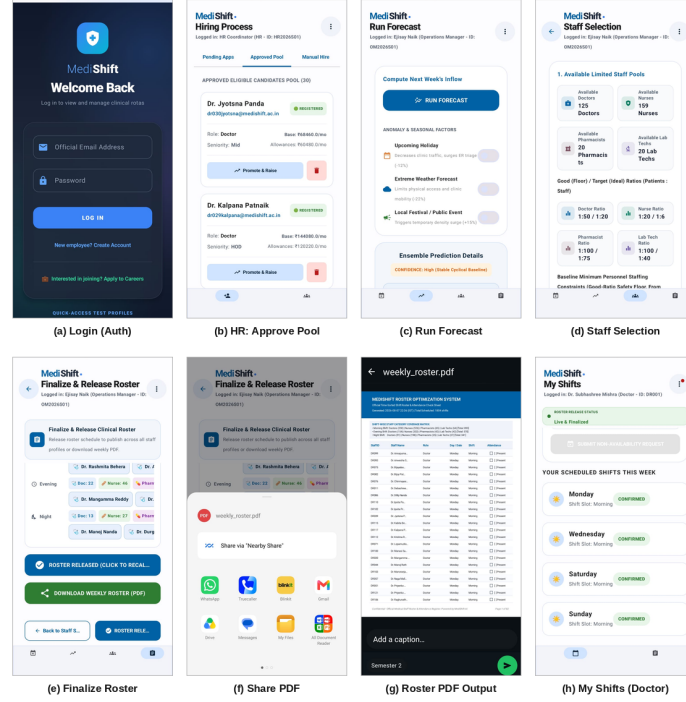


Figure 1.2: Representative MediShift screens spanning one full cycle: login, HR staff approval, forecasting and staffing on the Operations Manager’s device, roster release and PDF export, and the confirmed roster as seen by a doctor.

1.5 Data Source and Features

The forecasting engine learns from a dataset of hospital *shift records* spanning three years at three shifts a day (Morning, Evening, Night): 3,288 records in total. Each stores the date, day of week, shift type, actual patient inflow, weather, and whether the day was a holiday or local event. From these, MediShift builds a small set of numeric features: one-hot indicators for shift type, weather, holidays and local events; a 7-record rolling average of recent demand; a cyclic encoding of the calendar month for the boosting model (so December and January read as seasonal neighbours, not as 12 and 1); and a cyclic encoding of the day of week for both regression models, letting them learn a genuine Monday-versus-Friday pattern directly rather than only inferring it through the rolling average (Section 2.5).

One idea threads through every model in the next chapter: not every historical record deserves equal say. MediShift applies an *exponential recency weight* $w_i = d^{n-1-i}$ (decay $d = 0.5^{1/90}$) that halves a record’s influence roughly every 30 days, used directly inside Ridge Regression and the Gradient-Boosted Stumps. Table 1.1 lists the ten columns handed to the regression models, all genuinely known in advance of the shift being predicted.

Feature	What it captures
Intercept	Constant baseline term (always 1, Ridge Regression only)
Shift type	One-hot indicators for Morning / Evening / Night
Weather	One-hot indicator for adverse weather that day
Holiday flag	One-hot indicator for a public holiday
Local-event flag	One-hot indicator for a known local event
7-record rolling average	Mean inflow over the previous 7 shifts
Cyclic month encoding	cos transform of the month (boosting model only)
Cyclic day-of-week encoding	sin/cos transform of the weekday, Monday through Sunday (both regression models; Section 2.5)

Table 1.1: The feature set shared by Ridge Regression and the Gradient-Boosted Stumps model (Chapter 2). Ridge Regression uses ten features (intercept through day-of-week sine); the boosting model uses the same ten minus the intercept, which a tree-based model has no use for.

Chapter 2

Machine Learning and Predictions

2.1 Forecasting Approach

Predicting how many patients will arrive during tomorrow's Morning shift is, at heart, a question of learning from history. MediShift answers it with three models, each with its own strength (a regularised linear model, a boosted-tree ensemble, a classical time-series smoother), then blends their forecasts into one number rather than betting on a single technique. This chapter derives each model, states the formula MediShift computes, and closes with how the three are combined and judged.

Why not a neural network, the default choice for newer systems? Fit for purpose: a neural network needs far more than 3,288 shift records to outperform simpler models, its training is too slow and unstable for an on-device refit on every new data point, and its predictions are harder for a hospital administrator to inspect and trust. Ridge Regression, boosted stumps and Holt-Winters were chosen instead: each is well understood, refits on a phone in well under a second, and produces a forecast whose reasoning (a coefficient, a split rule, a trend line) can be read and questioned directly – which matters where a wrong, unexplainable forecast has real consequences.

2.2 Linear and Ridge Regression

The simplest starting point is to assume patient inflow is a straight-line combination of known features (is it a Morning shift, is it raining, was there a local event) plus some irreducible noise:

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon = \mathbf{x}^\top \boldsymbol{\beta} + \varepsilon. \quad (2.1)$$

Fitting this model means choosing the coefficient vector $\boldsymbol{\beta}$ that minimises the total squared error across every historical record, $\text{SSE}(\boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$. Setting its derivative to zero gives the classical closed-form solution, known as the *normal equations*:

$$\frac{\partial \text{SSE}}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = 0 \implies \boxed{\boldsymbol{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}} \quad (2.2)$$

Plain least squares treats every record as equally important and can chase small, accidental patterns in repetitive data, fitting the past closely but forecasting the future poorly. **Ridge Regression** fixes both issues. First, it adds a penalty $\alpha \sum_j \beta_j^2$ on the size of the coefficients, discouraging any one feature from dominating; minimising $(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \alpha \boldsymbol{\beta}^\top \boldsymbol{\beta}$ the same way as Equation (2.2) gives

$$\boldsymbol{\beta} = (\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}, \quad \alpha = 1.0. \quad (2.3)$$

Second, it applies the recency weight w_i from Section 1.5: scaling every row of $\mathbf{X}$ and $\mathbf{y}$ by $\sqrt{w_i}$ before solving Equation (2.3) is algebraically identical to weighting each squared error by w_i , so the fit leans on recent shifts far more than three-year-old ones. The feature vector

is the ten-column set of Table 1.1, including the cyclic day-of-week encoding of Section 2.5; once β is fitted, a forecast is $\hat{y} = \mathbf{x}^\top \beta$ evaluated on a feature vector built only from what is knowable in advance.

Worked example. Suppose Ridge Regression learns a (rounded) coefficient vector where the intercept is 18.2, the Morning indicator carries weight +9.4, the holiday flag +12.1, and the 7-record rolling average +0.35 per patient. For an ordinary Morning shift with a rolling average of 84 patients (other indicators at 0):

$$\hat{y} = 18.2 + 9.4(1) + 12.1(0) + 0.35(84) = 18.2 + 9.4 + 29.4 = 57.0.$$

On a public holiday, only that term switches on: $\hat{y} = 57.0 + 12.1 = 69.1$. This is the practical value of a linear model – each coefficient has a plain, inspectable meaning, so an administrator can see exactly which factor moved the forecast, unlike with a harder-to-interpret model.

2.3 Gradient-Boosted Decision Stumps

A straight line cannot capture every pattern; rain might only matter on a Monday. **Gradient boosting** instead fits a sequence of simple models, each correcting the mistakes left by those before it. MediShift uses *decision stumps*: the simplest tree, a single yes/no split on one feature. Starting from the recency-weighted mean F_0 , each of 30 rounds fits one stump to the current residuals $r_i = y_i - F_{m-1}(\mathbf{x}_i)$, choosing the feature and threshold that minimise weighted squared residual error on each side of the split, then adds it back at a small learning rate $\eta = 0.1$:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x}). \quad (2.4)$$

This is the same additive, residual-correcting idea libraries like XGBoost use, with shallower single-split trees and a simpler splitting rule – a deliberate trade that keeps it cheap enough to fit on a phone in under a second while staying a genuine, data-fitted ensemble rather than a fixed formula.

2.4 Time-Series Forecasting with Holt-Winters

Patient inflow is also a *time series*: it has a level, a trend, and a weekly seasonal rhythm. The best-known family for this is ARIMA, predicting a value from past values (autoregressive) and past forecast errors (moving-average), after removing trend by differencing. ARIMA is powerful, but fitting it well needs an iterative maximum-likelihood search rather than a single formula – too heavy for a model that should refit instantly on a phone. MediShift instead uses **Holt-Winters triple exponential smoothing**, modelling the same level/trend/season structure with simple, non-iterative update rules on every new shift logged:

$$L_t = \alpha(y_t - S_{t-m}) + (1 - \alpha)(L_{t-1} + T_{t-1}) \quad (2.5)$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1} \quad (2.6)$$

$$S_t = \gamma(y_t - L_t) + (1 - \gamma)S_{t-m} \quad (2.7)$$

with $\alpha = 0.35$, $\beta = 0.12$, $\gamma = 0.20$, and a seasonal cycle $m = 21$ (7 days $\times$ 3 shifts). A forecast h shifts ahead reads directly off the fitted state, $\hat{y}_{t+h} = L_t + hT_t + S_{t+h-m}$. Since every update

rule already blends in new data at a fixed rate, Holt-Winters weights recent shifts natively, with no separate weight array needed the way Sections 2.2 and 2.3 require. Its 21-slot seasonal array already spans a full week of day-of-week structure; evaluating it out to $h = 1, \dots, 21$ (Section 2.5), instead of stopping at the next occurrence of each shift type, turns this one formula into a forecast for every day of the roster week.

2.5 Ensemble Blending and Evaluation

Rather than trust each model equally, MediShift blends the three fitted forecasts by their own held-out accuracy, then combines that blend with a fast, transparent heuristic built from the recent trend, a smooth weekly cycle, and direct holiday/weather/event adjustments. Each model’s Mean Absolute Error (MAE, defined below) sets its own weight within a 60% fitted-model share, so a model genuinely more accurate on this hospital’s data earns more say instead of an equal three-way split:

$$\lambda_{\text{ridge}} = \frac{1/\text{MAE}_{\text{ridge}}}{\sum_m 1/\text{MAE}_m}, \quad \lambda_{\text{boosted}} = \frac{1/\text{MAE}_{\text{boosted}}}{\sum_m 1/\text{MAE}_m}, \quad \lambda_{\text{holt}} = \frac{1/\text{MAE}_{\text{holt}}}{\sum_m 1/\text{MAE}_m}, \quad (2.8)$$

where m ranges over the three fitted models and every MAE is floored at 0.5 patients, so an almost-perfect held-out fit cannot cause division by an implausibly small number. These weights, summing to 1 by construction, replace the flat one-third split in the blend:

$$\hat{y}_{\text{final}} = 0.6 \times (\lambda_{\text{ridge}} \hat{y}_{\text{ridge}} + \lambda_{\text{boosted}} \hat{y}_{\text{boosted}} + \lambda_{\text{holt}} \hat{y}_{\text{holt}}) + 0.4 \times \hat{y}_{\text{heuristic}}. \quad (2.9)$$

The fitted models keep the larger 60% share as the more reliable long-run predictors; the heuristic’s 40% share exists because holidays and local events are rare in the training data, so the fitted models have little to learn from them, while the heuristic reacts to an anomaly flag immediately. Figure 2.1 summarises the pipeline, and Algorithm 2.1 gives it as pseudocode.

From one shift to a full roster week. Equation (2.9) produces one blended forecast for whichever shift and day it is evaluated against. The roster in Chapter 3 needs one forecast for every day of its fixed Monday-through-Sunday template, not one day’s number copied across all seven – otherwise a single unusually busy or quiet day would silently set every day’s target. MediShift closes this gap two ways. Ridge Regression and the Gradient-Boosted Stumps model (Sections 2.2 and 2.3) are fitted on the cyclic day-of-week feature of Table 1.1, $\cos(2\pi\delta/7)$ and $\sin(2\pi\delta/7)$ for weekday index $\delta \in \{0, \dots, 6\}$ (Monday = 0), so each learns how a typical Monday differs from a typical Friday directly from three years of data, not only through the shared rolling average. Holt-Winters (Section 2.4) already carries this structure natively in its 21-slot seasonal array, simply evaluated out to $h = 1, \dots, 21$ instead of stopping at the next occurrence of each shift type.

The heuristic has no learned day-of-week signal of its own, so it is adjusted by an empirical factor from the day-level historical dataset of Section 1.5:

$$\phi_d = \text{clip}\left(\frac{\bar{y}_d}{\bar{y}}, 0.8, 1.25\right), \quad d \in \{\text{Mon}, \dots, \text{Sun}\}, \quad (2.10)$$

where $\bar{y}_d$ is the historical average on weekday d and $\bar{y}$ the overall average. The clip to $[0.8, 1.25]$ keeps a single outlier day from swinging its weekday’s target by more than about a quarter, in the same spirit as the recency window of Section 1.5; a weekday with fewer than three samples

stays at the neutral value $\phi_d = 1$. Equation (2.9) is then evaluated once per day d of the roster week, using day d 's own fitted-model forecasts and $\phi_d \cdot \hat{y}_{\text{heuristic}}$ in place of the plain heuristic term:

$$\hat{y}_d = 0.6 (\lambda_{\text{ridge}} \hat{y}_{\text{ridge},d} + \lambda_{\text{boosted}} \hat{y}_{\text{boosted},d} + \lambda_{\text{holt}} \hat{y}_{\text{holt},d}) + 0.4 (\phi_d \cdot \hat{y}_{\text{heuristic}}), \quad d \in \{\text{Mon}, \dots, \text{Sun}\}. \quad (2.11)$$

This is the forecast Chapter 3 actually uses: $\text{target}_{d,s}$ in Section 3.2 is derived from $\hat{y}_d$'s own shift-wise split for that specific day d , never from a single day's forecast reused across the week.

Every model's accuracy is reported on held-out records it never saw while fitting, using three standard measures. *Mean absolute error* is the plain average miss, in patients:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (2.12)$$

Root mean squared error squares each miss before averaging, then takes the square root back, so a few large misses are punished more than the same total error spread thinly across many small ones – relevant since one badly missed shift can matter more operationally than several off by one or two patients:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (2.13)$$

R^2 asks a different question: not how big the miss is, but how much better the model is than the simplest guess, the historical average $\bar{y}$:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}. \quad (2.14)$$

An R^2 of 1 is a perfect fit, 0 means no better than the average, and a negative R^2 – reported rather than hidden – means the model does worse than that simplest guess.

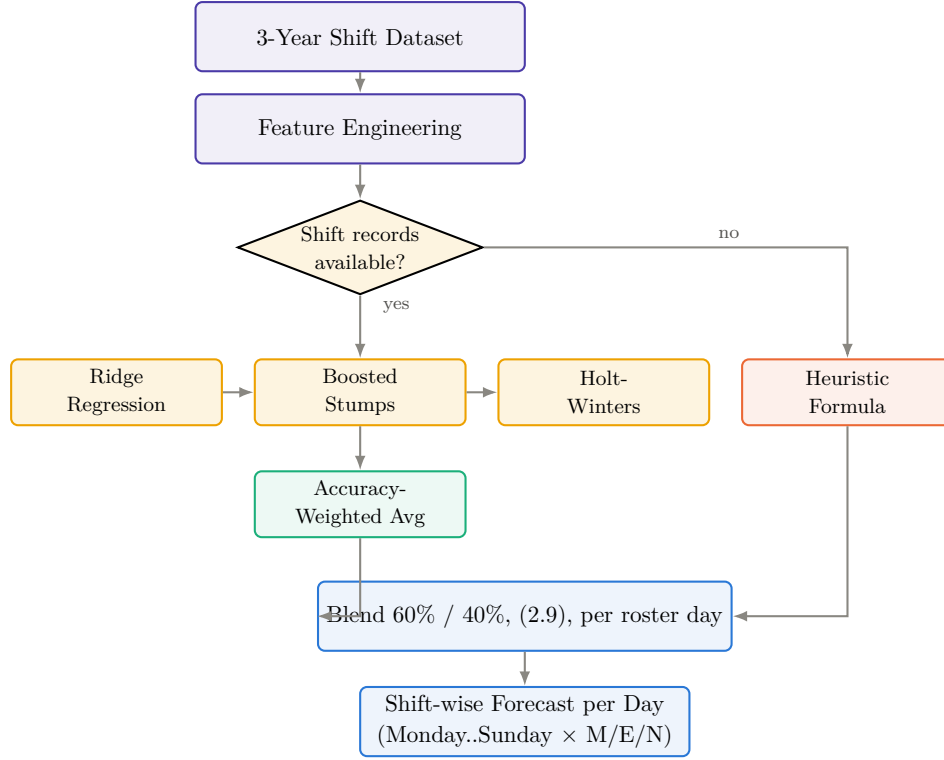


Figure 2.1: Forecasting pipeline. When shift-level history is available, three fitted models are accuracy-weighted, combined, and blended with the day-of-week-adjusted heuristic once for every day of the roster week; otherwise the heuristic alone drives the forecast.

Algorithm 2.1 Blended per-day forecast for the roster week

Require: historical shift records R , day-level history H , anomaly flags (holiday, weather, local event)

- 1: $h \leftarrow \text{HEURISTICFORECAST}(R, \text{flags})$ ▷ recency + momentum + season + anomaly
 - 2: $\{\phi_d\} \leftarrow \text{day-of-week factors from } H$ ▷ Equation (2.10)
 - 3: **if** per-shift dataset available **then**
 - 4: $r \leftarrow \text{FITRIDGE}(R)$; $g \leftarrow \text{FITBOOSTEDSTUMPS}(R)$; $s \leftarrow \text{FITHOLTWINTERS}(R)$
 - 5: $\{\lambda_{\text{ridge}}, \lambda_{\text{boosted}}, \lambda_{\text{holt}}\} \leftarrow \text{inverse-MAE weights}$ ▷ Equation (2.8)
 - 6: **for** each roster day $d \in \{\text{Mon}, \dots, \text{Sun}\}$ **do**
 - 7: $f_d \leftarrow 0.6 (\lambda_{\text{ridge}} r_d + \lambda_{\text{boosted}} g_d + \lambda_{\text{holt}} s_d) + 0.4 (\phi_d \cdot h)$ ▷ Equation (2.11)
 - 8: **end for**
 - 9: **else**
 - 10: **for** each roster day $d \in \{\text{Mon}, \dots, \text{Sun}\}$ **do**
 - 11: $f_d \leftarrow \phi_d \cdot h$ ▷ fallback when no shift-level history is available
 - 12: **end for**
 - 13: **end if**
 - 14: **return** $\{f_d\}$, each split into Morning / Evening / Night shares
-

Chapter 3

Staff Selection and Roster Optimisation

Chapter 2 produced a number for every day of the rostering week: how many patients are expected in Morning, Evening and Night, Monday through Sunday (Equation (2.11)), not one figure copied across all seven days. This chapter explains how that week of numbers becomes an actual roster: which named doctor, nurse, pharmacist or lab technician works which shift, on which day, while respecting the ratio policy from Section 1.1 and every rest and leave rule the hospital enforces.

3.1 The Two-Ratio Staffing Model

For a predicted patient count p and a staff category c (Doctor, Nurse, Pharmacist, Lab Technician), two headcounts are computed from the Good ratio g_c and Target ratio t_c introduced in Section 1.1:

$$\text{minSafe}_c = \left\lceil \frac{p}{g_c} \right\rceil, \quad \text{ideal}_c = \left\lceil \frac{p}{t_c} \right\rceil, \quad (3.1)$$

the hard safety floor and the aspirational target respectively. The staffing decision for category c is then an integer program: choose a headcount n_c as close as possible to ideal_c without dropping below minSafe_c or exceeding the available pool,

$$\text{minimise } |n_c - \text{ideal}_c| \quad \text{subject to} \quad \text{minSafe}_c \leq n_c \leq \text{pool}_c, \quad n_c \in \mathbb{Z}. \quad (3.2)$$

This has a short, exact derivation. Every category's objective and constraints involve only its own variable, so the four categories solve independently; and since $t_c \leq g_c$ always holds, $\text{ideal}_c \geq \text{minSafe}_c$, so the closest point to ideal_c inside $[\text{minSafe}_c, \text{pool}_c]$ is simply

$$\boxed{n_c^* = \min(\text{ideal}_c, \text{pool}_c)} \quad (3.3)$$

Staff each category as close to ideal as the pool allows, and flag a genuine shortage only if even pool_c falls short of minSafe_c . Since Morning, Evening and Night see very different patient loads, this calculation runs separately per shift block on that shift's own predicted count, so a quiet Night shift's headcount is never propped up by Morning's larger numbers, nor its own safety floor left unchecked.

Worked example. Suppose a Morning shift forecast is $p = 96$ patients, and nurse ratios are $g_{\text{nurse}} = 6$ (Good), $t_{\text{nurse}} = 4$ (Target):

$$\text{minSafe}_{\text{nurse}} = \left\lceil \frac{96}{6} \right\rceil = 16, \quad \text{ideal}_{\text{nurse}} = \left\lceil \frac{96}{4} \right\rceil = 24.$$

With 20 nurses in the pool, $\text{pool}_{\text{nurse}} = 20$ lies between the two, so $n_{\text{nurse}}^* = \min(24, 20) = 20$: every available nurse works, clearing the safety floor of 16 with four to spare, short of the 24-nurse ideal only because the pool is limited. With 30 nurses instead, $n_{\text{nurse}}^* = \min(24, 30) = 24$: the ideal is met exactly, and six nurses stay free for another shift or category. Only below 16 nurses would MediShift raise a genuine understaffing flag.

3.2 Per-Shift Allocation and Person-Level Assignment

Equation (3.3) says how many staff of each category a shift needs. Turning that into a roster of actual names is a harder, combinatorial problem: the same pool of people must split fairly across all three shifts of a day and all seven days of the week without anyone being double-booked, overworked, or denied rest. MediShift formulates this as a binary integer program.

Sets, parameters and decision variable.

C	the four staff categories (Doctor, Nurse, Pharmacist, Lab Tech.),
$I_c = \{1, \dots, m_c\}$	staff members of category $c \in C$ (I below means I_c),
$D = \{1, \dots, 7\}$	the seven days of the rostering week,
$S = \{M, E, N\}$	shift types (Morning, Evening, Night),
$\text{target}_{d,s} \in \mathbb{Z}_{\geq 0}$	ideal headcount for shift s on day d (Eq. (3.3)),
$L_{i,d} \in \{0, 1\}$	1 if staff member i has approved leave on day d ,
$x_{i,d,s} \in \{0, 1\}$	1 if staff member i works shift s on day d .

$\text{target}_{d,s}$ is indexed by day d deliberately, not just shift s : each day draws on its own entry in the weekly forecast $\{\hat{y}_d\}$ of Equation (2.11), so a busier Monday and a quieter Sunday get their own targets rather than one day's forecast standing in for the whole week.

Objective. The person-level assignment minimises total deviation from every shift's target headcount, summed across the whole week,

$$\text{minimise } \sum_{d \in D} \sum_{s \in S} \left| \text{target}_{d,s} - \sum_{i \in I} x_{i,d,s} \right|, \quad (3.4)$$

linear in any actual solver after the standard substitution of the absolute value by a non-negative slack $u_{d,s} \geq \text{target}_{d,s} - \sum_i x_{i,d,s}$ and $u_{d,s} \geq \sum_i x_{i,d,s} - \text{target}_{d,s}$, minimising $\sum u_{d,s}$.

Hard constraints. Five rules are enforced exactly and never traded away, even if a shift then falls short of its ideal headcount:

$$\sum_{s \in S} x_{i,d,s} \leq 1 \quad \forall i \in I, d \in D, \quad (3.5)$$

$$\sum_{d \in D} \sum_{s \in S} x_{i,d,s} \leq 5 \quad \forall i \in I, \quad (3.6)$$

$$x_{i,d,s} \leq 1 - L_{i,d} \quad \forall i \in I, d \in D, s \in S, \quad (3.7)$$

$$x_{i,d,N} + x_{i,d+1,M} + x_{i,d+1,E} \leq 1 \quad \forall i \in I, d \in D \setminus \{7\}, \quad (3.8)$$

$$\sum_{k=0}^3 x_{i,d+k,N} \leq 3 \quad \forall i \in I, d \in \{1, 2, 3, 4\}. \quad (3.9)$$

(3.5) caps each person at one shift a day; (3.6) caps the week at five shifts; (3.7) removes anyone on approved leave from that day's shifts; (3.8) is the rest rule: a Night shift on day d blocks only Morning and Evening the next day, so consecutive Night shifts are allowed, but the day that run ends is necessarily a full rest day, since Morning and Evening are both shut

and no Night shift is left to take. (3.9) is the night-block cap: every rolling four-day window $\{d, d+1, d+2, d+3\}$ allows at most three Night shifts for the same person, so nobody is ever rostered onto a fourth consecutive one. A flat per-person weekly total, $\sum_{d \in D} x_{i,d,N} \leq 3$, would also rule out two separate, fully-rested three-night blocks in the same week – the rolling-window form above is what actually captures "not *continuously* more than three," not "not more than three in total."

These five rules are the *only* thing Objective (3.4) is constrained by: it is minimised subject to (3.5)–(3.9) and nothing else.

A fully certified optimal solver for a problem this size, coupling every day's decision to the next through Constraint (3.8), would be too slow to run interactively on a phone. MediShift instead uses a constructive two-step procedure that satisfies Constraints (3.5)–(3.9) exactly and closes the deviation in Objective (3.4) as tightly as the available pool allows.

Step one splits a day's available pool of a category fairly across its three shifts before naming anyone, using the **largest-remainder method**: each shift's exact proportional share of the pool is computed, the whole-number part is given to every shift, and any leftover units go one at a time to whichever shifts have the largest fractional remainder, so no shift ends up more than one person away from its fair share.

Worked example. Suppose Wednesday's available nurse pool is 22, and the ratio-matching model wants 13 nurses for Morning, 10 for Evening and 6 for Night (Section 3.1), 29 in total, more than the pool holds. Each shift's exact share of the 22 nurses, proportional to its demand, is

$$22 \times \frac{13}{29} = 9.86, \quad 22 \times \frac{10}{29} = 7.59, \quad 22 \times \frac{6}{29} = 4.55.$$

Rounding down gives 9, 7 and 4: 20 nurses placed, 2 left over. The two spare nurses go to the largest fractional remainders, Morning (0.86) then Evening (0.59), leaving Night's smaller remainder unfilled. The split is **Morning 10, Evening 8, Night 4**: all 22 nurses placed, each shift within one person of its exact fair share.

When one category's forecast is disproportionately weighted toward the later shifts of the week, applying this split independently to every day can starve the last day or two of the week's own available pool, since earlier days are filled first. MediShift guards against this with a weekly-capacity-aware scaling factor: each day's ideal headcount is scaled down slightly whenever the category's cumulative weekly demand would otherwise exceed what the week's pool can actually sustain, so the shortfall is spread evenly across the week instead of landing entirely on the final days.

Step two names staff for each shift's now-known headcount in up to three passes: a preference-aware ranking (favouring an active night-shift streak, then day-off and shift preferences, then workload balance); a fallback pass that drops only the preference ranking; and a rare emergency pass for very small pools. All three passes still enforce every hard rule above without exception – a person already on a third consecutive Night shift, for instance, is removed from the Night candidate pool in every pass, not just the first. Figure 3.1 and Algorithm 3.1 show the full sequence.

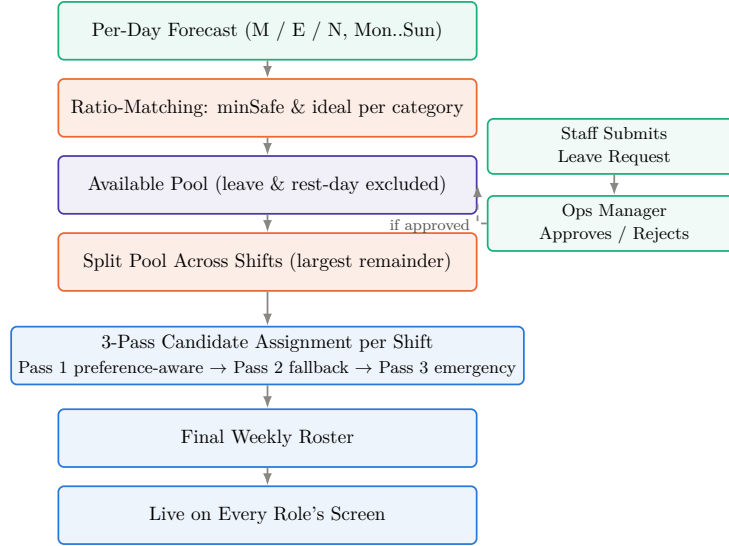


Figure 3.1: Roster generation pipeline. Approved leave removes a staff member from the available pool before the shift split and candidate search ever run.

Algorithm 3.1 Per-day roster assignment

```

1: for each day  $d$  of the week do
2:   for each category  $c$  do
3:      $\text{pool}_c \leftarrow$  staff of category  $c$  not on rest or approved leave for  $d$ 
4:      $\{p_s\} \leftarrow$  day  $d$ 's own forecast  $\hat{y}_d$ , split by shift  $s$  ▷ Equation (2.11)
5:      $\{\text{minSafe}_s, \text{ideal}_s\} \leftarrow$  Equation (3.3) for each shift  $s$ 
6:      $\{\text{target}_{c,s}\} \leftarrow \text{LARGESTREMAINDER\_SPLIT}(\text{pool}_c, \text{minSafe}, \text{ideal})$ 
7:   end for
8:   for each category  $c$  do
9:     for each shift  $s$  do
10:      assign top-ranked eligible staff up to  $\text{target}_{c,s}$  ▷ Pass 1: preference-aware
11:      if still short then assign remaining eligible staff by fewest shifts so far ▷ Pass 2
12:      end if
13:      if still short then relax night-block preference only, hard rules unchanged ▷
14:      Pass 3
15:      end if
16:    end for
17:  end for
18: return final roster
  
```

Optimality verification report. Since Algorithm 3.1 is constructive rather than a certified solver, MediShift lets the Operations Manager check its work rather than trust it blindly: an *Optimality Verification Report* that re-scans the released roster against Constraints (3.5)–(3.9) independently of the algorithm that produced it, counting how many instances of each rule apply and how many actually hold, and naming any specific violation it finds. This turns "the solver enforces hard rules" from a claim about the code into a number the Operations Manager can check after every roster generation.

3.3 Leave Management and Real-Time Integration

A staff member requests leave from a card on their own screen; an Operations Manager approves or rejects it. Approval turns it into a hard constraint ($x_{i,d,s} = 0$ for every shift on every approved day), enforced at every stage of Section 3.2’s pipeline – the pool count, all three assignment passes – so approved leave can never be worked around, even when the emergency pass has to run. A rejected request changes nothing. Thanks to the single-source-of-truth design of Section 1.4, an approval changes one stored value and every screen watching the staff pool, including the next generated roster, reflects it immediately.

3.4 Algorithms at a Glance

Table 3.1 collects every algorithm from both engines with its purpose and computational cost, for a reader auditing the system.

Algorithm	Type	Purpose	Cost
Ridge Regression	Closed-form linear	Recency-weighted forecast	$O(np^2 + p^3)$
Gradient-Boosted Stumps	Additive ensemble	Residual correction	$O(n \cdot p \cdot \text{rounds})$
Holt-Winters	Exp. smoothing	Level/trend/season forecast	$O(n)$
Ensemble Blend	Weighted average	Combine 3 models	$O(1)$
Ratio-Matching Model	Closed-form IP	Category headcount	$O(1)$ per category
Largest-Remainder Split	Rounding heuristic	Fair pool split across shifts	$O(k \log k)$
Roster Assignment	Constructive	Person-shift assignment	$O(\text{days} \times \text{staff} \log \text{staff})$
Optimality Verification	Rule-based audit	Post-hoc check of (3.5)–(3.9)	$O(\text{days} \times \text{staff})$

Table 3.1: Algorithms used by MediShift’s forecasting and staffing engines. n = historical records, p = feature count, k = shifts per day.

3.5 Future Work

Certified solver and shared backend. Google OR-Tools’ CP-SAT, the natural certified solver for the MILP in Section 3.2, has no Android build. The fix is a small Python backend: it would run CP-SAT under a bounded time limit and return the optimal roster with a proven optimality gap, something the current constructive procedure cannot report. The same backend could also host the app’s data behind a RESTful API (FastAPI), turning the on-device Room database into a local cache and letting multiple devices share one roster. Since every database call already goes through the single `Repository` class (Section 1.4), this would not touch the `ViewModel` or any UI screen.

Richer forecasting and field validation. A properly fitted ARIMA/SARIMA model could sharpen forecasts for hospitals with longer historical records, and a field study comparing patient wait times and staff overtime before and after adoption would test MediShift’s real-world impact directly.

3.6 Conclusion

MediShift replaces a manual, spreadsheet-based process with one automated pipeline: forecasting patient demand and converting it directly into a safe, fair staff roster that stays current as leave is approved. Three genuinely fitted models, Ridge Regression, Gradient-Boosted Stumps, and Holt-Winters smoothing, blend into each day's forecast; a closed-form ratio-matching model (Section 3.1) turns that into a headcount; and a constraint-respecting constructive search turns it into a named roster.

Every part was checked, not assumed. The forecasting models (Chapter 2) were evaluated on held-out data using MAE, RMSE and R^2 . The staffing algorithm (Chapter 3) enforces every hard rule in Section 3.2 by construction, so violations are structurally impossible, and its closed-form result (Equation (3.3)) was hand-verified against worked examples.

In short: a phone-only system, built honestly and checked carefully, already does meaningfully better than a manually built weekly roster.

References

1. World Health Organization. *Workload Indicators of Staffing Need (WISN): User's Manual*. WHO Press, Geneva.
2. *Hospital Workforce Demographics & Ratio Report* and *Professional Staffing Report*, internal hospital workforce reference documents reviewed for this project, summarising recommended patient-to-staff ratios and observed staffing gaps in government and district hospitals.
3. Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
4. Hyndman, R.J. and Athanasopoulos, G. *Forecasting: Principles and Practice*. OTexts.
5. Cormen, T.H., Leiserson, C.E., Rivest, R.L., and Stein, C. *Introduction to Algorithms*. MIT Press.
6. Chen, T. and Guestrin, C. *XGBoost: A Scalable Tree Boosting System*. KDD 2016.
7. Anthropic. *Claude (Large Language Model)*. Used for vibe-coding parts of the Android application and for brainstorming design and algorithmic ideas.
8. OpenAI / Google, et al. *Large Language Model based AI tools*. Used for formatting this report and refining its language into a clearer, more professional style.

Tools Used: **Android Studio** (For Major Task), **Google AI Studio** (For Architecture Initialization), **Claude AI** (Vibe Coding, Fine Tuning, Bug Fixing), **Google Gemini**(Minor Clarifications), **Grammarly**

Special Thanks to Mr. Vishal, Mr. Saurabh Chaudhary, Mr. Harshit Marskole for providing clarity on different parts of the project